

## EXAMEN PRÁCTICO DE LA UNIDAD I – TIPO 1

SI-988 Soluciones Móviles II | FixTrack

Estudiante: Sergio Colque Ponce (2022073503)

Repositorio: <https://github.com/diegocastillo12/fixtrack-mobile>

Docente: Dr. Óscar Juan Jiménez Flores

### 1. Validación de formulario y feedback

La pantalla valida que el correo tenga estructura válida y que la contraseña tenga seis o más caracteres. Form.validate detiene el flujo antes de contactar Supabase. Durante la solicitud, \_isLoading deshabilita el botón y muestra CircularProgressIndicator. Los errores se muestran en la misma vista.

Archivos: app/lib/features/auth/presentation/login\_page.dart

[INSERTAR captura del código, error de validación en emulador y spinner activo]

### 2. Bloqueo de intentos

LoginController lleva failedAttempts en la lógica de negocio. Después de tres respuestas rechazadas, el cuarto intento retorna antes de AuthGateway.signIn, muestra bloqueo durante 30 segundos y no envía una petición. Al terminar el plazo, el contador se reinicia.

Archivos: app/lib/features/auth/domain/login\_controller.dart

[INSERTAR captura de signIn con contador y corte antes del servicio]

### 3. Cierre de sesión ante HTTP 401

DioClient detecta 401 solo cuando la petición llevaba Authorization. Llama al callback onUnauthorized y termina sin entrar en el bloque de reintentos. El callback borra auth\_token de SharedPreferences, cierra la sesión local de Supabase y navega al login, eliminando las rutas anteriores.

Archivos: app/lib/core/network/dio\_client.dart;

app/lib/features/incidencias/data/incidencia\_repository\_impl.dart

[INSERTAR captura del interceptor y del callback de cierre]

### 4. Tres pruebas unitarias del login

Las pruebas a, b y c verifican el estado inicial sin sesión, la autenticación exitosa con token guardado y el rechazo con una sola llamada al backend falso.

Archivos: app/test/examen\_tipo1\_test.dart

[INSERTAR captura de terminal con comando y resultado PASS real]

### 5. Bloqueo y respuesta HTTP 401

La prueba d comprueba cero llamadas adicionales en el cuarto intento. La prueba e inyecta 401 mediante un HttpClientAdapter falso y verifica que se borra el token, se cierra la sesión y se envía una sola petición.

Archivos: app/test/examen\_tipo1\_test.dart

[INSERTAR captura de código completo de pruebas d y e y terminal PASS real]

Ejecución de pruebas

Desde la raíz del repositorio:

```
cd app flutter test test/examen_tipo1_test.dart
```

Estado de verificación: pendiente de ejecutar en una estación con Flutter SDK. No se incluyen resultados ni capturas simuladas.

Código completo de las pruebas

```
import 'dart:convert';
```

```
import 'dart:typed_data';
```

```
import 'package:app/core/network/dio_client.dart';
```

```
import 'package:app/features/auth/domain/login_controller.dart';
```

```
import 'package:dio/dio.dart';
```

```
import 'package:flutter_test/flutter_test.dart';
```

```
void main() {
```

```
  group('Examen Tipo 1', () {
```

```
    test('a: estado inicial sin sesión', () {
```

```
      final login = LoginController(gateway: _AuthFake(), tokens: _TokensFake());
```

```
      expect(login.status, LoginStatus.unauthenticated);
```

```
      expect(login.failedAttempts, 0);
```

```
    });
```

```
    test('b: login exitoso guarda token y autentica', () async {
```

```
      final auth = _AuthFake();
```

```
      final tokens = _TokensFake();
```

```

    final login = LoginController(gateway: auth, tokens: tokens);
    expect(await login.signIn(' usuario@upt.pe ', 'secreto'), isTrue);
    expect(auth.calls, 1);
    expect(auth.lastEmail, 'usuario@upt.pe');
    expect(login.status, LoginStatus.authenticated);
    expect(tokens.token, 'token-de-prueba');
  });

  test('c: login rechazado registra error sin reintento', () async {
    final auth = _AuthFake(reject: true);
    final login = LoginController(gateway: auth, tokens: _TokensFake());
    expect(await login.signIn('usuario@upt.pe', 'invalida'), isFalse);
    expect(login.status, LoginStatus.error);
    expect(auth.calls, 1);
  });

  test('d: el cuarto intento bloqueado genera cero llamadas', () async {
    final auth = _AuthFake(reject: true);
    final login = LoginController(gateway: auth, tokens: _TokensFake());
    for (var i = 0; i < 3; i++) {
      expect(await login.signIn('usuario@upt.pe', 'invalida'), isFalse);
    }
    final callsBeforeFourth = auth.calls;
    expect(await login.signIn('usuario@upt.pe', 'invalida'), isFalse);
    expect(auth.calls - callsBeforeFourth, 0);
    expect(login.status, LoginStatus.blocked);
    expect(login.error, contains('Espera 30 segundos'));
  });

  test('e: HTTP 401 borra token y no reintenta', () async {
    final tokens = _TokensFake()..token = 'token-de-prueba';
    var sessionActive = true;
    final client = DioClient(
      baseUrl: 'http://localhost:4010',
      tokenProvider: () async => sessionActive ? tokens.token : null,
      onUnauthorized: () async {
        await tokens.clear();
        sessionActive = false;
      },
    );
    final adapter = _UnauthorizedAdapter();
    client.dio.httpClientAdapter = adapter;
    await expectLater(client.dio.get<dynamic>('/recurso'),
      throwsA(isA<DioException>()));
    expect(tokens.token, isNull);
    expect(sessionActive, isFalse);
    expect(adapter.calls, 1);
    expect(adapter.authorization, isTrue);
  });
}

class _AuthFake implements AuthGateway {
  _AuthFake({this.reject = false});
  final bool reject;
  int calls = 0;
  String? lastEmail;

  @override
  Future<String> signIn(String email, String password) async {
    calls++;
    lastEmail = email;
  }
}

```

```

        if (reject) throw StateError('Credenciales rechazadas');
        return 'token-de-prueba';
    }
}

class _TokensFake implements TokenStore {
    String? token;

    @override
    Future<void> save(String value) async {
        token = value;
    }

    @override
    Future<void> clear() async {
        token = null;
    }
}

class _UnauthorizedAdapter implements HttpClientAdapter {
    int calls = 0;
    bool hadAuthorization = false;

    @override
    Future<ResponseBody> fetch(RequestOptions options,
        Stream<Uint8List>? requestStream, Future<void>? cancelFuture) async {
        calls++;
        hadAuthorization = options.headers.containsKey('Authorization');
        return ResponseBody.fromString(jsonEncode({'error': 'unauthorized'}), 401,
            headers: {'content-type': ['application/json']});
    }

    @override
    void close({bool force = false}) {}
}

```